

Documentação Fluxo de Dados

Origem e Estrutura dos Dados (Raw)

Os dados brutos utilizados no projeto foram consumidos de arquivos CSV armazenados em um catálogo do ambiente Databricks nomeado como "Landing". Abaixo estão detalhados os arquivos da origem, suas colunas originais e um exemplo real extraído de cada arquivo.

1. catalogo_produtos.csv

- Dados cadastrais de produtos.
- Colunas: id_produto, nome_produto, categoria, preco, fornecedor, peso_kg, estoque_disponivel, avaliacao_interna, ativo, data_cadastro_produto
- Exemplo de linha: PROD-0001, Smartphone, 128GB, Eletronicos, 2504.00, Shenzhen Connect Brasil, 2.99, 429, null, Nao, 2024-04-10

2. suporte_tickets.csv

- Registros de atendimentos ao cliente.
- Colunas: ticket_id, id_cliente, id_pedido, tipo_problema, data_abertura, data_resolucao, tempo_resolucao_horas, agente_suporte, nota_avaliacao
- Exemplo de linha: 33d4eede-c2c6-52be-a8d3-eea55f6fa56e, d3719c5c-4134-4820-aabd-0c5dffc55034, 810622ab-18c1-47bc-b3d6-e5d0316be981, pro, 2023-01-03T05:13:00.000Z, 2023-01-11T20:13:00.000Z, 58.3, Thiago Alves, 4

3. clientes.csv

- Base cadastral de usuários.
- Colunas: id_cliente, nome, sobrenome, email, telefone, genero, data_nascimento, data_cadastro, endereco, cidade, estado, pais, device_ids, origem
- Exemplo de linha: 0d244537-8164-4b84-bcf2-0e43766f3221, Nathalia, LUZ, nathalia_luz@hotmail.com, 14.9977.1729 r. 282, F, 1988-05-31, 2023-11-09, 107 Joseph Station, Jaguaribara, Ceará, Brasil, 56f3a452-b3f5-488b-b90d-8cb2b3343c70;3ab7d21b-48dd-44f8-88ae-c99a4dd51ef1;faced7cc-4d83-4505-9d86-497626fe16d0;dd49cd20-eacc-49f1-b4b5-92551e77d916, web

4. pedidos.csv

- Transações de vendas.
- Colunas: id_pedido, id_cliente, id_produto, valor_pedido, data_pedido, metodo_pagamento, status, quantidade
- Exemplo de linha: 0000336c-e8de-435d-876e-df51705f648f, 7b652fe2-c341-47a7-9830-0d4f3c726c03, PROD-0430, 550.66, 2024-11-11, PIX, Aprovado, 2

5. avaliacoes.csv

- Feedback de produtos e NPS.
- Colunas: id_avaliacao, id_pedido, id_cliente, id_produto, nota_produto, comentario, nota_nps, recomenda, data_avaliacao
- Exemplo de linha: 00000000-0000-0000-0f8f-c3a92f8a32c5, 0000336c-e8de-435d-876e-df51705f648f, 7b652fe2-c341-47a7-9830-0d4f3c726c03, PROD-0430,3, "Minha filha gostou, mas acho que poderia ter mais peças.", 7, S, 2024-11-17 00:00:00

6. clickstream.csv

- Logs de navegação e eventos digitais.
- Colunas: id_evento, id_sessao, id_cliente, id_dispositivo, id_produto, tipo_evento, canal, dispositivo, origem_sessao, data_evento, tempo_pagina_seg
- Exemplo de linha: 000018f7-e26f-4a90-b68b-59e21f598917, sess-023070, 7d7e8650-175e-4ef3-a5ee-368412f7ba71, de2ab406-cd40-4599-8e4a-9add47a8bf72, , log_in, web, desktop, social, 2023-01-14T22:33:46.000Z, 563

Ingestão de Dados (Landing → Bronze)

O processo de ingestão da camada Bronze foi construído através do notebook Landing-Bronze.ipynb focando em fidelidade aos dados brutos, idempotência e rastreabilidade. A ingestão não aplica regras de negócio, servindo como o histórico imutável da entrada dos dados.

A abordagem técnica implementada baseia-se nos seguintes pilares:

- **Configuração Declarativa por Tabela:** Em vez de inferir arquivos dinamicamente, o pipeline utiliza um dicionário de configuração (TABLE_CONFIG). Isso garante controle estrito sobre o que é ingerido, mapeando explicitamente o nome do arquivo (/Volumes/stack_overnol/default/landing), o nome da tabela de destino, separadores, encoding e se o arquivo é de presença obrigatória ou opcional.
- **Leitura:** Os arquivos CSV são lidos com as opções header=True e inferSchema=True. Adicionalmente, ativamos multiLine=True (para suportar quebras de linha em campos de texto) e usamos o mode="PERMISSIVE", garantindo que linhas malformadas não derrubem o pipeline ou sejam descartadas, mas sim mantidas para tratamento posterior na camada Silver.
- **Sanitização Automática de Colunas:** Como o Delta Lake possui restrições quanto a caracteres especiais em nomes de colunas (espaços, vírgulas, chaves, etc.), foi implementada uma função de limpeza via Regex (sanitize_column_names) que padroniza os cabeçalhos automaticamente sem intervenção manual.
- **Enriquecimento com Metadados (Rastreabilidade):** Nenhuma coluna original é alterada, mas cada linha recebe metadados de auditoria cruciais para a linhagem dos dados:
 - `_src_file`: Caminho de origem do arquivo CSV.
 - `_ingested_at` e `_ingestion_date`: Timestamp de controle padronizado para toda a execução do pipeline.
 - `_row_hash`: Um hash MD5 de todas as colunas concatenadas, essencial para detecção de mudanças (CDC) e deduplicação nas camadas seguintes.
- **Persistência Otimizada em Delta Lake:** Os DataFrames enriquecidos são gravados no Unity Catalog (schema stack_overnol.bronze) no formato Delta (format("delta")).
 - **Idempotência:** A gravação utiliza o modo overwrite combinado com overwriteSchema=True, garantindo que reexecuções do pipeline sejam seguras, não gerem duplicatas e propaguem automaticamente evoluções de schema.

- **Particionamento:** Os dados físicos são particionados pela coluna de metadado `_ingestion_date`, otimizando a performance para leituras futuras.
- **Observabilidade e Tolerância a Falhas:** O pipeline não é interrompido caso uma única tabela falhe. Ele captura as exceções e, ao final, exibe um log estruturado em formato tabular detalhando o status (Success/Failed), o tempo de execução, os erros encontrados e a volumetria de linhas ingeridas por tabela.

Camada de Purificação (Bronze -> Silver)

O Condicional de Idempotência (Coluna “ja_tratada”): Em todas as tabelas do notebook, a primeira etapa após a leitura da camada Bronze é a verificação: `if "ja_tratada" in df.columns:`

O que isso faz e porque garante a idempotência dos dados? Isso atua como um mecanismo de fast-pass (atalho). Se os dados provenientes da camada anterior já possuírem a flag “ja_tratada”, o pipeline entende que as regras de negócio e limpezas já foram aplicadas. Em vez de reprocessar todas as expressões regulares, casts e joins pesados, ele simplesmente seleciona as colunas finais e regravava. Isso garante idempotência (rodar o pipeline 1 ou 100 vezes gera o mesmo resultado) e otimiza severamente os custos de computação em caso de reexecuções. Esse processo foi adicionado para que uma automação envolvendo comunicação dos dados do backend com o Databricks se tornasse possível e eficiente mesmo com limitações de ferramentas e de tempo.

Detalhamento das Transformações por Tabela:

1. Clientes (silver.clientes)

A primeira tabela foca em higienizar dados cadastrais sensíveis e aplicar validações lógicas de localização.

- **Filtro:** Remoção de registros onde o `id_cliente` é nulo.
- **Origem:** Normalizada estritamente para Web, App, Indicação ou Outro.
- **Telefone e Ramal:** Foi aplicada uma Regex para extrair possíveis ramais (números após “r” ou “r.”) para uma nova coluna `ramal`. O telefone principal foi limpo (remoção do DDI 55 e caracteres não numéricos) e reformatado com uma máscara condicional: (XX) XXXX-XXXX ou (XX) XXXXX-XXXX.

- **E-mail:** Convertido para minúsculas. Foi feita a correção inteligente de provedores (ex: preenchimento de @ caso faltasse em gmail, hotmail, etc.) e a remoção de duplicidade de arrobas (@@).
- **Nome e Sobrenome:** Padronizados com a primeira letra maiúscula (initcap).
- **Gênero:** Mapeamento de diversas strings (como "m", "male", "masc") para valores absolutos: Masculino, Feminino ou Não Informado.
- **Datas (Nascimento e Cadastro):** Uso da função coalesce associada ao try_to_date para tentar mapear e converter múltiplos formatos de string (yyyy-MM-dd, dd/MM/yyyy, etc.) em uma data real (DateType).
- **Estado e Cidade (Validação Cruzada):** Foi criada uma lista de validação (estados_validos). Caso um dado de "cidade" estivesse preenchido na coluna de "estado" e vice-versa, o script inverte os valores automaticamente, corrigindo o erro de preenchimento.
- **Deduplicação:** Drop de duplicatas com base no id_cliente.

2. Dispositivos por Cliente (silver.clientes_dispositivo)

Esta tabela não vem de um arquivo .csv como as demais, mas sim de uma modelagem relacional a partir dos dados da tabela cliente.

- **Desmembramento (Explode):** A coluna original device_ids (que continha vários aparelhos na mesma linha separados por ;) passou por um split seguido de um explode. Isso multiplicou as linhas, gerando um registro único (uma linha) para cada dispositivo atrelado àquele cliente.
- **Limpeza:** Aplicação de trim e filtro para remover dispositivos que ficaram vazios ("") após a quebra.
- **Deduplicação:** Garantia de unicidade através da chave composta id_cliente + id_dispositivo.

3. Catálogo de Produtos (silver.catalogo_produtos)

Foco na tipagem de valores financeiros e sanitização de categorias.

- **Filtro Obrigatório:** Remoção de id_produto nulo.
- **Status Ativo:** Strings diversas indicando positividade ("s", "sim", "yes", "1") viraram o booleano True. Negativas viraram False.
- **Categoria:** Coluna limpa de números e símbolos. Foi aplicado um forte processo de categorização: caso a categoria estivesse ausente ou confusa, o script analisa o próprio nome_produto com Regex (rlike buscando "shampoo", "impressora", etc.) para inferir e forçar a categoria correta ("Beleza", "Eletrônicos", etc.).
- **Pesos e Estoque:** Conversão de strings com a palavra literal "null" para valores lógicos (Nulo matemático para peso, 0 para estoque).

- **Preço:** Remoção da string "R\$" e espaços vazios, substituição de vírgulas por pontos e conversão para tipo double. Valores menores ou iguais a zero foram anulados.

4. Trilha de Ações do Usuário (silver.clickstream)

Tabela de eventos de navegação.

- **Filtros:** Eliminação de linhas sem id_evento ou id_cliente.
- **Tipo e Canal de Evento:** Normalização de strings (substituindo espaços/traços por _). Criação da categoria canal_padronizado ("Web", "App" ou "Outros").
- **Dispositivo e Origem de Sessão:** Agrupamento de variações de aparelhos ("celular", "mob") para Mobile e padronização da origem de tráfego ("Social", "Orgânico", "Busca Paga", "Direto", "Email").
- **Tempo de Página:** Tratamento para impedir valores negativos (convertidos para Nulo).
- **Data:** Tipagem reforçada garantindo o formato timestamp.
- **Deduplicação:** Remoção de duplicatas na chave primária id_evento.

5. Pedidos (silver.pedidos)

Tabela de pedidos que traz valores financeiros.

- **Filtro Obrigatório:** Remoção de id_pedido ou id_cliente nulos.
- **Data do Pedido:** Mesmo processo de coalesce da tabela de clientes, suportando diversos formatos de input e convertendo para tipo Data.
- **Valor do Pedido:** Limpeza intensa de caracteres da moeda local ("R\$"), padronização decimal com ponto e exclusão lógica de valores ≤ 0 (substituídos por None).
- **Método de Pagamento e Status:** Uso de regex para manter apenas caracteres alfanuméricos e *case conditions* para unificar variações ortográficas em categorias limitadas ("Pix", "Boleto", "Cartão") e ("Aprovado", "Cancelado").
- **Quantidade:** Tratamento de numerais que porventura vieram escritos em extenso e cast final para tipo Integer.

6. Avaliações (silver.avaliacoes)

Tratamento das métricas qualitativas e de satisfação.

- **Notas de Produto e NPS:** Tratamento avançado convertendo classificações em texto ("ótimo", "péssimo") para seus equivalentes numéricos. Além disso, foram implementadas barreiras lógicas (Limites): a nota_produto não pode exceder 5 nem ser menor que 1, e o NPS foi restringido ao limite entre 0 e 10.

- **Data da Avaliação (Enriquecimento):** Formatos de data foram padronizados. Se a avaliação viesse sem data de preenchimento, foi feito um JOIN com a tabela de Pedidos para herdar a data de quando o pedido foi efetuado, garantindo que o registro não fique órfão no tempo.
- **Deduplicação:** Tratamento de duplicidade via id_avaliacao.

7. Tickets de Suporte (silver.suporte_tickets)

Tabela focada no cálculo de métricas de atendimento (SLAs).

- **Tipo de Problema:** Variações textuais (ex: "delay", "entr", "p3oduto") foram higienizadas e agrupadas em silos analíticos: "Produto", "Pagamento", "Entrega", "Reembolso".
- **Tempo de Resolução:** A matemática utilizada nessa coluna foi converter os timestamps de data_abertura e data_resolucao para o padrão UNIX (segundos desde 1970). A diferença entre os dois foi dividida por 3600 para obter dinamicamente a métrica calculada de tempo_resolucao_horas em formato Inteiro.
- **Deduplicação:** Exclusão de duplicatas pela chave ticket_id.

Camada de Negócios (Silver -> Gold)

A camada Gold tem como responsabilidade consolidar, agregar e cruzar os dados purificados da camada Silver, transformando-os em tabelas analíticas orientadas a responder perguntas de negócio.

Para manter a solução simples, de alta performance e pronta para consumo direto por dashboards e agentes de IA, a modelagem adotou duas convenções arquiteturais claras:

- **Dimensões “dim_”:** Tabelas descritivas e cadastrais.
- **Data Marts “dm_”:** Tabelas desnormalizadas e pré-agregadas para domínios específicos.
-

1. Dimensão de Clientes (gold.dim_cliente)

Tabela descritiva que atua como o cadastro unificado de clientes.

- **Objetivo:** Fornecer os atributos qualificadores (quem é, onde mora, perfil sociodemográfico) para agrupamentos e filtros em dashboards.
- **Origem:** Derivada diretamente da silver.clientes.

- **Tratamento:** Seleção das colunas higienizadas e criação de chaves únicas de identificação para fácil junção com os data marts.

2. Dimensão de Produtos (gold.dim_produto)

Tabela do catálogo de produtos oferecidos.

- **Objetivo:** Centralizar as características estáticas e de categorização dos produtos.
- **Origem:** Derivada da silver.catalogo_produtos.
- **Tratamento:** Consolidação de atributos como nome, categoria higienizada, fornecedor e preço base, servindo como o eixo de análise para o mix de produtos da empresa.

3. Visão Cliente 360 (gold.dm_cliente_360)

Um Data Mart desnormalizado focado no comportamento completo do usuário.

- **Objetivo:** Responder rapidamente: *"Quem são os nossos melhores clientes? Qual o nível de satisfação deles? Quantos problemas eles tiveram?"*.
- **Cruzamentos (Joins):** Agrega dados da silver.clientes, silver.pedidos, silver.avaliacoes e silver.suporte_tickets.
- **Métricas Consolidadas:**
 - Comportamento de Compra: Contagem total de pedidos, ticket médio, valor total gasto ao longo da vida (LTV - *Lifetime Value*).
 - Engajamento e Satisfação: Média do NPS dado pelo cliente e total de avaliações feitas.
 - Atrito: Quantidade de tickets de suporte abertos por aquele cliente específico.

4. Visão Produto 360 (gold.dm_produto_360)

Um Data Mart focado no desempenho comercial e operacional de cada item do catálogo.

- **Objetivo:** Responder: *"Quais produtos vendem mais? Quais geram mais tickets de suporte? Quais têm a melhor ou pior avaliação?"*.
- **Cruzamentos (Joins):** Agrega silver.catalogo_produtos, silver.pedidos, silver.avaliacoes e silver.suporte_tickets.
- **Métricas Consolidadas:**
 - Performance de Vendas: Quantidade total de unidades vendidas e receita bruta gerada por produto.
 - Qualidade: Média final da nota_produto (limitada de 1 a 5 na Silver).

- Operação: Estoque disponível contrastado com o volume de vendas e o total de tickets de suporte atrelados a defeitos/problemas daquele produto específico.

5. Vendas por Período (gold.dm_vendas_periódodo)

Um Data Mart de série temporal otimizado para análises financeiras e de tendências.

- **Objetivo:** Analisar a saúde financeira da empresa ao longo do tempo (diário, mensal, anual) e sazonalidades.
- **Origem Principal:** silver.pedidos (focando em status = "Aprovado").
- **Métricas Consolidadas:**
 - Agrupamento temporal pela data_pedido.
 - Somatório de receita (GMV), contagem de transações e métodos de pagamento mais utilizados naquele período.

Otimização de Performance

Todas as tabelas citadas acima receberam o comando OPTIMIZE com ZORDER BY nas suas chaves primárias ou colunas de filtro mais comuns), garantindo que as consultas analíticas sejam processadas com a maior eficiência possível.

Banco de Dados Local e Job Workflow

Por que foi escolhido exportar como SQLite em vez de PostgreSQL para o banco local

Optamos pelo SQLite no ambiente local fundamentalmente por conta de sua arquitetura *serverless* e alta portabilidade (o banco inteiro é compactado num único arquivo). Se utilizasse o PostgreSQL, a equipe e os validadores do projeto precisariam de *overhead* infraestrutural (como subir um container Docker, configurar usuários e portas de rede). Com o SQLite, entregamos rapidamente a capacidade de homologação e conexão com a solução proposta que contém dashboards e agente de IA, de forma super leve e zero-setup, permitindo que a solução seja avaliada e reproduzida sem fricção, mantendo o foco analítico da camada Ouro intacto. O banco SQLite em questão foi tanto disponibilizado no drive quanto é possível ser criado ao rodar o script `seed.py` que está dentro da pasta `backend/bd` do repositório.

Job Workflow

Por fim, foi configurada toda a orquestração do fluxo em um job cluster de performance otimizada.

- Cron/Agendamento: O workflow roda todos os dias à meia-noite e com fuso horário brasileiro.
- Dependências de Pipeline (DAG): Criei um encadeamento seguro onde:
 1. A tarefa Landing_to_Bronze busca o notebook base.
 2. A tarefa Bronze_to_Silver tem a dependência da Landing_to_Bronze para rodar, protegendo o tratamento.
 3. A tarefa final Silver_to_Gold também depende estritamente do sucesso da Bronze_to_Silver. Se uma fase quebra, o Data Lake inteiro fica protegido de inserções erradas.

